

# Projekt: “Analizator danych z elementami uczenia maszynowego w Pythonie”

## Cel projektu:

Celem projektu jest pokazanie umiejętności praktycznego zastosowania składni języka Python oraz elementów omawianych na zajęciach m.in.:

- dekompozycji kodu (funkcje, moduły, klasy),
- obsługi wyjątków,
- wykorzystania kolekcji,
- programowania obiektowego,
- pracy z bibliotekami (NumPy, Pandas, Matplotlib, sklearn),
- oraz zasad czytelności i modularności kodu.

Projekt nie wymaga w pełni działającej aplikacji (jest to opcja dodatkowa) — **ważniejsze jest pokazanie poprawnej struktury i koncepcji programu.**

Kod powinien być czytelny, dobrze opisany i pokazywać znajomość języka.

## Zakres techniczny projektu:

### 1. Wprowadzenie do języka i składni

- Użycie podstawowych typów danych, pętli, warunków i operacji wejścia/wyjścia.
- Przykład: menu tekstowe, które pozwala użytkownikowi wybrać rodzaj analizy.

### 2. Dekompozycja i wyjątki

- Podział projektu na logiczne funkcje i klasy.
- Obsługa błędów wejścia, braków danych, błędnych formatów plików.
- Przykład: `try/except FileNotFoundError, ValueError`, własne wyjątki `InvalidDataError`.

### 3. Kolekcje i importowanie

- Dane przechowywane w listach, słownikach lub zestawach.
- Import modułów standardowych i własnych (np. `import utils.data_loader`).

### 4. Programowanie obiektowe i przeciążanie operatorów

- Klasy reprezentujące dane lub modele, np. `Dataset`, `Report`, `Model`.
- Przeciążenie operatora `__add__` (np. łączenie dwóch raportów) lub `__str__` (czytelny wydruk).

## 5. Generatory i context managery

- Generator do przetwarzania dużych zbiorów danych krok po kroku.
- Context manager do obsługi plików lub połączeń z API (`with open(...) as f`).

## 6. Dekoratory i zaawansowane OOP

- Dekorator do logowania czasu wykonania funkcji (`@measure_time`) lub błędów (`@safe_run`).
- Dziedziczenie klas, np. `BaseModel`  $\rightarrow$  `LinearRegressionModel`, `ClassifierModel`.

## 7. Wzorce projektowe (Dekomp. II)

- Zastosowanie jednego wzorca projektowego, np.
  - **Factory** – tworzenie różnych modeli analitycznych,
  - **Singleton** – konfiguracja aplikacji,
  - **Strategy** – wybór algorytmu analizy.

## 8. Moduły i pakiety (Dekomp. III)

- Struktura projektu w stylu:

```
projekt/
├─ main.py
├─ data/
│   └─ loader.py
│   └─ validator.py
├─ analysis/
│   └─ statistics.py
│   └─ models.py
├─ utils/
│   └─ logger.py
│   └─ decorators.py
├─ tests/
│   └─ test_analysis.py
```

## 9. NumPy

- Operacje na macierzach, np. obliczanie średnich, wariancji, korelacji, macierzy kowariancji.

## 10. TDD i testy automatyczne

- Testy funkcji obliczeniowych (`pytest` lub `unittest`).

- Przykład: test funkcji `mean()` lub działania modelu predykcyjnego.

## 11. Pandas

- Wczytywanie danych z pliku `.csv`, filtrowanie, grupowanie, czyszczenie braków.
- Tworzenie raportu z wynikami.

## 12. Scikit-learn

- Implementacja prostego modelu:
  - regresja liniowa (`LinearRegression`),
  - klasyfikacja (`KNeighborsClassifier`),
  - klasteryzacja (`KMeans`).

## 13. Matplotlib

- Wizualizacja wyników analizy lub modelu ML (np. wykres rozrzutu, histogram, wykres trendu).

---

## Wymagania końcowe:

- Projekt uruchamiany z pliku `main.py`.
- Kod zgodny ze standardem PEP8.
- Dokumentacja w pliku `README.md` (cel, sposób uruchomienia, opis modułów).
- Minimum 2 testy jednostkowe.
- Minimum 1 wykres z Matplotlib.
- Minimum 1 model ML z `sklearn`.
- Minimum 1 własny dekorator lub context manager.

| Kryterium                                | Opis                                                | Punkty |
|------------------------------------------|-----------------------------------------------------|--------|
| Działający projekt                       | Program uruchamia się bez błędów i wykonuje analizę | 30     |
| Dekompozycja i OOP                       | Kod podzielony na klasy/funkcje, logiczny podział   | 20     |
| Obsługa błędów                           | Try/Except, walidacja danych, komunikaty            | 10     |
| NumPy / Pandas / Matplotlib / Sklearn    | Poprawne wykorzystanie bibliotek                    | 25     |
| Testy / dekoratory / wzorce / generatory | Wykorzystanie zaawansowanych narzędzi               | 10     |
| Dokumentacja i czytelność kodu           | README, komentarze, zgodność z PEP8                 | 5      |